

LangChain构建聊天机器人



黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌



学习目标

Learning Objectives

1. 了解LangChain框架作用
2. 掌握LangChain框架的安装
3. 了解LangChain框架的组成
4. 掌握LangChain框架的基本使用
5. 综合案例：Streamlit + LangChain打造GPT聊天机器人

01

LangChain概述

LangChain框架介绍

2023年可以说是AI大语言模型发展元年，随着OpenAI的ChatGPT和GPT-4的发布，点燃了人工智能大语言模型的发展浪潮，各大科技公司纷纷推出了自家的大语言模型产品，各国更是将大语言模型的发展作为人工智能技术的重要突破来推进，纷纷进行业务和技术层面的布局。

然而现有预训练大模型也存在一定技术缺陷，如仅有通用知识表示，知识只截止在训练时的日期，对于新知识无从习得。而大模型的更新训练需要耗费巨量的资源和成本，对于大模型应用层面的企业或个人更是无力承担。

因此在这种背景下，出现了LangChain开源框架，用于将外部数据和LLM大语言模型相结合，让LLM能够基于外部数据知识的基础上进行智能问答，这就是LangChain开源框架搭建的基于AI大模型下的AI应用开发新范式。

LangChain框架介绍

LangChain是一个用于开发由语言模型驱动的应用程序的框架，旨在帮助开发人员使用语言模型构建端到端的应用程序。它提供了一套工具、组件和接口，可简化创建由大型语言模型（LLM）和聊天模型提供支持的应用程序的过程。

LangChain 可以轻松管理与语言模型的交互，将多个组件链接在一起，可连接其他数据源，即LangChain是一个可以将大语言模型和外部数据及计算相结合的开源框架，可以让大语言模型能够基于外部数据源进行数据生成。

一个强大而又有差异的应用程序不仅会通过 API 调用语言模型，也会将语言模型连接到其他数据源进行数据感知（如 Pinecone），还允许语言模型能够与其环境进行交互（如访问互联网获取数据），而LangChain可以很好的完成这些任务。目前LangChain开源框架提供了Python和JavaScript包。

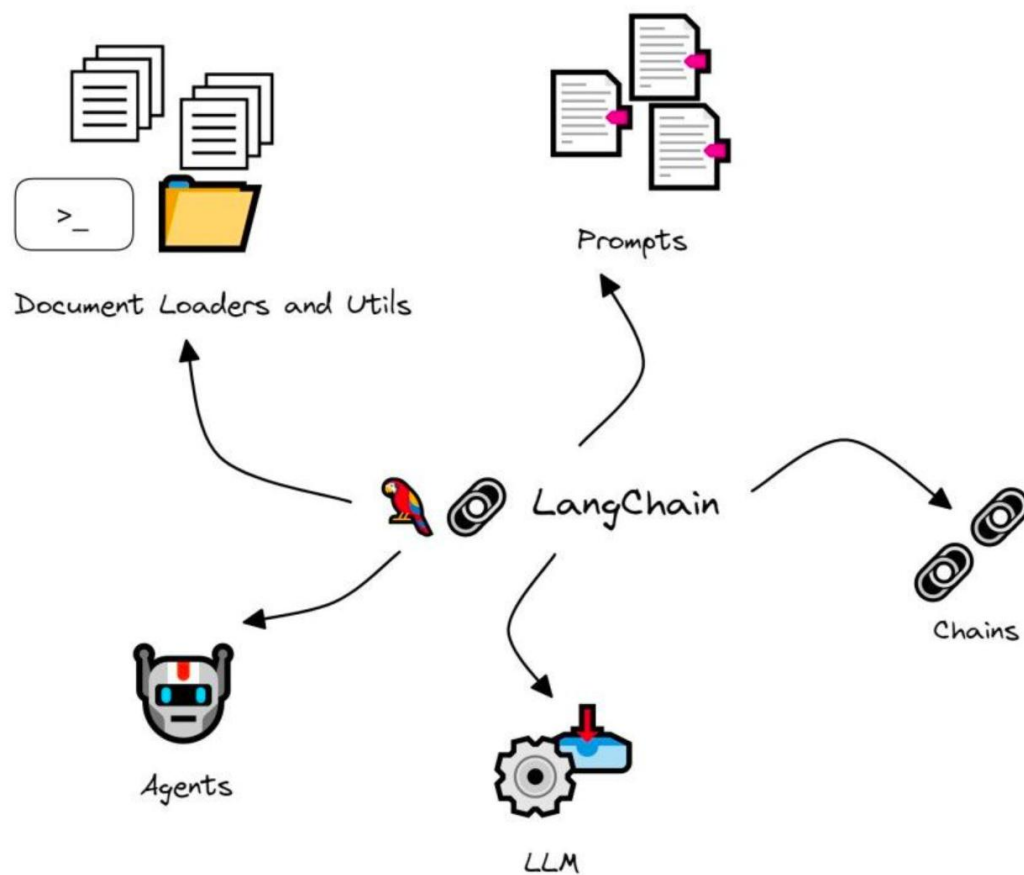
LangChain框架组成

与大模型不断涌现的业务创新路线不同，LangChain选择了一条基于大模型之上的应用开发框架创新之路，它借助大模型出色的多模态数据理解和生成能力，结合外部计算和数据源提供应用程序开发的框架，并且对代码进行开源，使得其在2023年3月随着GPT-4的发布其关注度得到了直线飙升。

LangChain定义了一套基于大模型之上的应用开发框架，倡议各大模型能够按照框架定义进行相关实现，其也在不断完善其在大模型上的支持的能力，它的更新迭代很快，新的内容在不断出现。

LangChain框架组成

下图描述了LangChain的业务架构，内部核心业务包括六个部分：Prompts、Models、Chains、Memory、Retriever、Agent。



LangChain框架组成

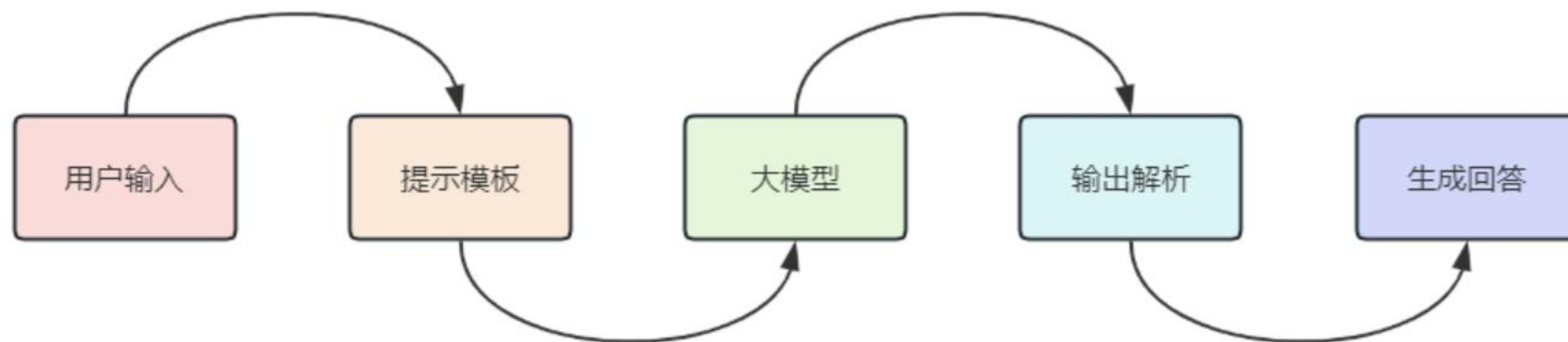
Prompts: 用于定义与模型交互的方式。通过Prompts，可以创建与LLM模型交互的模板，从而实现与模型的交互。

LangChain框架组成

Model模型（LLM）：提供语言理解和生成能力是AI应用的核心

LangChain框架组成

Chains链：这个组件是把不同组件串联起来的结构链，能让我们创建出复杂的流程，流程里的每个组件可以负责处理特定的任务。

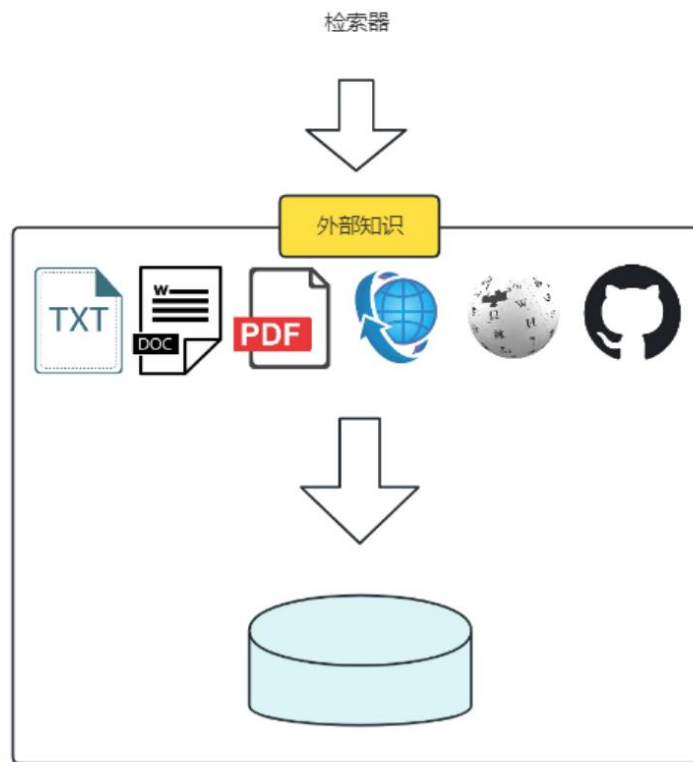


LangChain框架组成

Memory记忆：主要用来储存和管理对话历史或相关的上下文信息，这个组件是对话型AI应用中保持连贯性和上下文感知的关键。

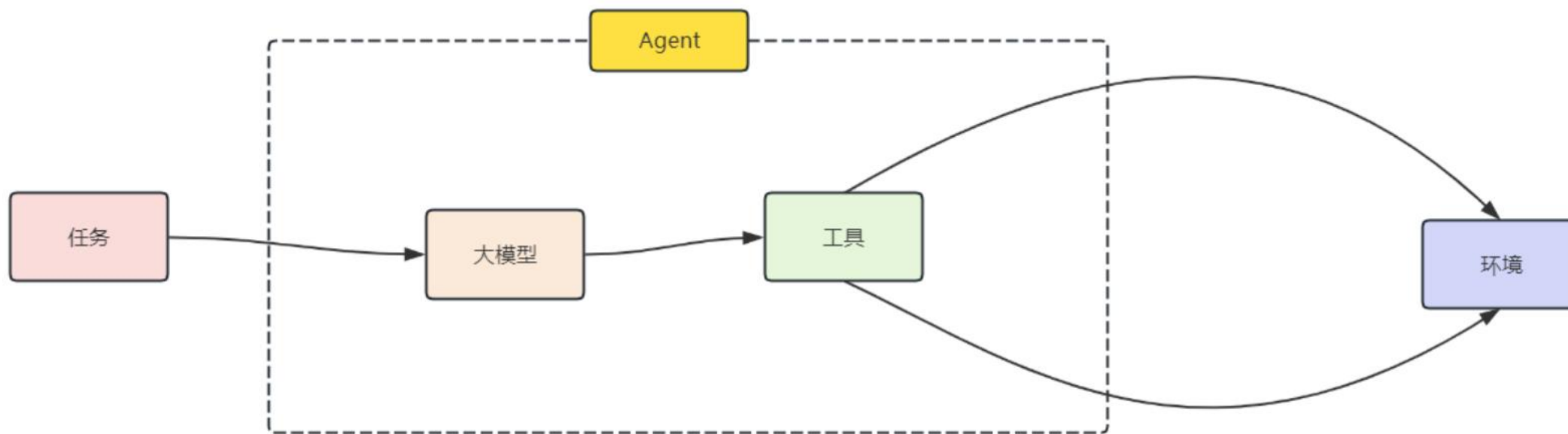
LangChain框架组成

Loader加载器（Retriever检索器）：用于从指定源加载数据，支持多种数据源，如文件夹、Azure存储、CSV文件、印象笔记、Google网盘、任意的网页、PDF、S3等。



LangChain框架组成

Agent智能体：代表一个基于大模型的能执行一系列动作的智能体，它的核心理念是利用AI模型的能力进行推理，根据任务动态评估和确定行动路径。





02

LangChain基础使用

安装LangChain及相关包

```
pip install langchain -i https://pypi.tuna.tsinghua.edu.cn/simple
```

```
pip install langchain-community -i https://pypi.tuna.tsinghua.edu.cn/simple
```

```
pip install dashscope -i https://pypi.tuna.tsinghua.edu.cn/simple
```

获取阿里百炼API KEY

官网地址: <https://tongyi.aliyun.com/>



获取阿里百炼API KEY

大模型服务平台百炼

阿里云百炼于近期正式发布2.0版本，应用、模型双引擎全新升级，兼容LlamaIndex等开源框架，更具开放性的应用框架，更极致生态化的模型服务，降低推理成本，加速大模型应用落地。

立即开通

获取专属服务

快捷入口

控制台

文档

API

SDK

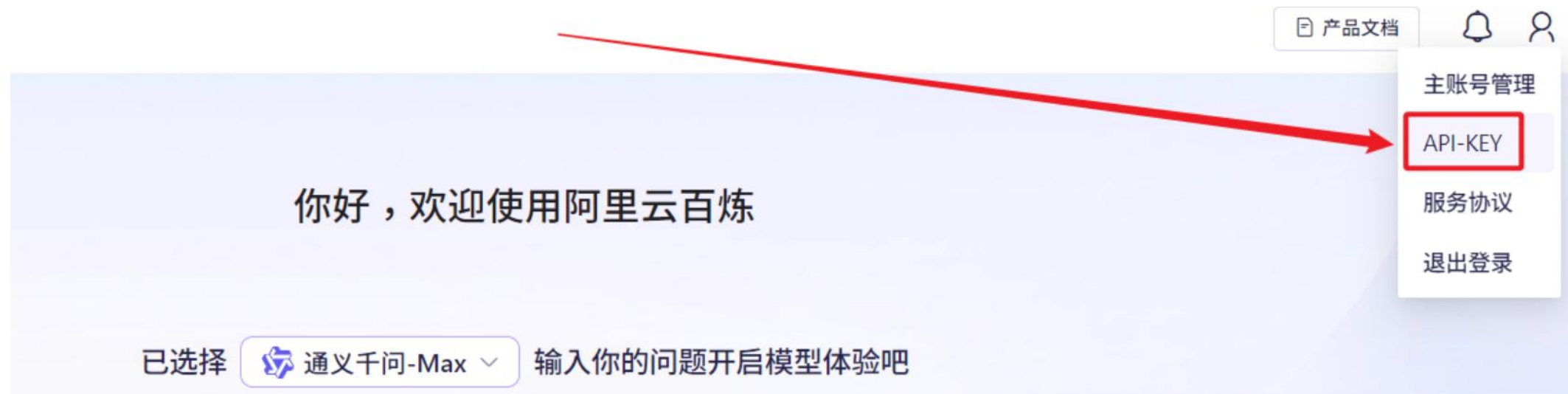
产品定价



01:02

了解阿里云百炼

单击右上角个人中心，找到API-KEY，如下图所示：



你好，欢迎使用阿里云百炼

已选择  通义千问-Max 输入你的问题开启模型体验吧

产品文档

- 主账号管理
- API-KEY
- 服务协议
- 退出登录

获取阿里百炼API KEY

创建API KEY



API KEY

sk-7****281d

创建新的API-KEY

* 归属业务空间: 主账号空间

描述: Tongyi大模型

9 / 200

取消 确定

创建时间

2024-07-09 23:58:55

申请Qwen模型使用权限

应用创建

模型调用

结合业务需求，选择适合的大模型，支持API/SDK的调用方式



立即调用 →

查看教程 ▶

应用创建

基于Assistant API，支持调用Prompt、插件、RAG等多种能力，完成大模型应用的创建。



立即创建 →

查看教程 ▶

申请Qwen模型使用权限

Prompt

Prompt优化

测试版本

发布版本

在这里输入

知识检索增强 ☐

长期记忆 ? Beta 申请

插件

流程 Beta

继续使用百炼大模型服务，需要开通以下商品，并创建模型调用API-KEY

- 使用大模型服务会按计费规则进行计费，需要开通以下商品，开通后即可使用相关服务，按照对应服务定价产生计费，不使用则不会产生计费。
- 如下列举了当前计费的模型。未来新增模型计费会直接更新到您已开通的商品中，您无需再次开通。
- 新增模型计费或修改模型计费，均会在控制台进行公告，请您在调用模型前留意模型单价和计费规格。

百炼大模型推理 (146)

百炼大模型部署 (21)

百炼大模型训练 (14)

通义千问-Max (qwen-max)

FLUX-schnell (flux-schnell)

通义千问-Max-0428 (qwen-max-0428)

Qwen-Long (qwen-long)

通义千问2-开源版-72B (qwen2-72b-instruct)

通义千问2-开源版-57B (qwen2-57b-a14b-instruct)

通义千问2-开源版-7B (qwen2-7b-instruct)

通义千问2-开源版-1.5B (qwen2-1.5b-instruct)

通义千问2-开源版-0.5B (qwen2-0.5b-instruct)

FLUX-dev (flux-dev)

通义千问-Max-20K (qwen-max-longcontext)

☒ 我已阅读并同意《模型管理服务协议》

计费详情

取消开通

确认开通

获取API KEY

API KEY获取地址: <https://bailian.console.aliyun.com/?apiKey=1#/api-key>

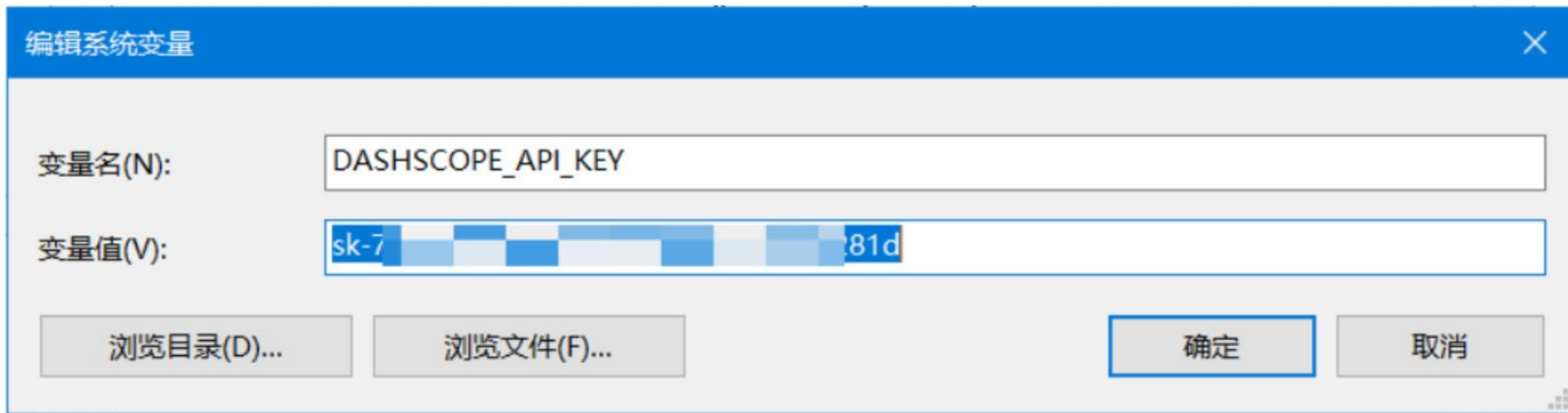
我的API-KEY

创建我的API-KEY (2/10)

API KEY	所归属业务空间	描述	创建时间	操作
sk-7****281d	默认业务空间 (llm-e9cb6l679st7nfw0)	传智大模型	2024-07-09 23:58:55	查看 删除
sk-37a1:7ff79f	默认业务空间 (llm-e9cb6l679st7nfw0)	Tongyi千问大模型	2024-07-24 09:32:43	复制 删除

将获取的环境变量添加到环境变量

Windows环境变量设置:



The image shows the 'Edit System Variables' dialog box in Windows. The title bar is blue with the text '编辑系统变量' and a close button. The main area has a light gray background. There are two text input fields: '变量名(N):' with the value 'DASHSCOPE_API_KEY' and '变量值(V):' with the value 'sk-7...81d'. Below these fields are four buttons: '浏览目录(D)...', '浏览文件(F)...', '确定', and '取消'. The '确定' button is highlighted with a blue border.

注：变量名称必须为DASHSCOPE_API_KEY

案例

文本生成案例

需求：基于LangChain调用Tongyi，实现文本生成（文本扩写），Prompt：从前有一个美丽的小村庄，那里的人们过着宁静的生活。

案例

文本问答案例

需求：基于LangChain调用Tongyi，实现文本问答，Prompt：世界上最高的山峰是哪一座？

案例

代码生成案例

需求：基于LangChain调用Tongyi，实现代码生成，Prompt：请为以下功能生成Python代码：功能：计算两个数的最大公约数（GCD）

调用常见问题说明

常见问题总结：

第一种情况：忘记配置环境变量DASHSCOPE_API_KEY

第二种情况：环境没有生效（明明配置了环境变量，但是跑不起来），首次配置，重启PyCharm或计算机

第三种情况：提前在阿里云进行实名认证而且一定要提前开通通义模型权限（qwen-max）

03

ChatPromptTemplate模板

AI模型支持的消息模板类

```
from langchain.prompts import ChatPromptTemplate
prompt_template = ChatPromptTemplate.from_messages(
    [("system", "..."), ("human", "..."), ("ai", "...")]
)
```

> system

系统消息

> human

用户消息类

> ai

模型消息类



案例

基于LangChain的翻译工具

需求：基于LangChain中的ChatPromptTemplate构建一个翻译器，英语翻译汉语

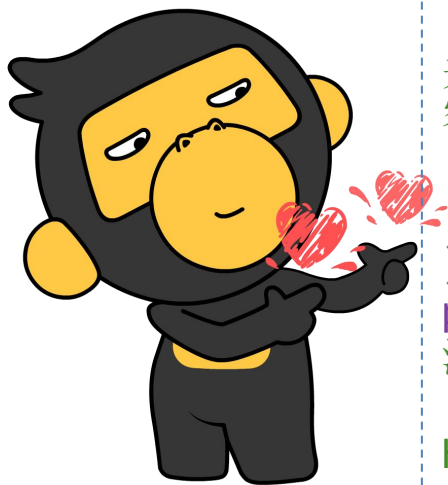
Prompt: I'm so hungry I could eat a horse

 案例

基于LangChain的翻译工具

最终代码:

```
from langchain_community.llms import Tongyi
from langchain.prompts import ChatPromptTemplate
prompt_template= ChatPromptTemplate.from_messages([
    ("system", "你是一位专业的翻译，能够将{input_language}翻译成{output_language}，并且输出文本会根据用户要求的任何语言风格进行调整。请只输出翻译后的文本，不要有任何其它内容。"),
    ("human", "文本: {text}\n语言风格: {style}"),
])
prompt_value=prompt_template.invoke({"input_language": "英语", "output_language": "汉语",
                                     "text": "I'm so hungry I could eat a horse", "style": "文言文"})
llm= Tongyi(model_name='qwen-max')
result=llm.invoke(prompt_value)
print(result)
```



04

打造GPT聊天机器人

最终效果图



utils工具箱

```
from langchain.chains import ConversationChain
from langchain_community.llms import Tongyi
import os
from langchain.memory import ConversationBufferMemory
def get_chat_response(prompt, memory, api_key):
    model= Tongyi(model="qwen-max", dashscope_api_key=api_key)
    chain= ConversationChain(llm=model, memory=memory)
    response=chain.invoke({"input":prompt})
    return response["response"]

# memory = ConversationBufferMemory(return_messages=True)
# print(get_chat_response("牛顿提出过哪些知名的定律?", memory,
# os.getenv("DASHSCOPE_API_KEY")))
# print(get_chat_response("我上一个问题是什么?", memory,
# os.getenv("DASHSCOPE_API_KEY")))
```



main主界面

```
import streamlit as st
from langchain.memory import
ConversationBufferMemory
from utils import get_chat_response
st.title("🗨️ 克隆ChatGPT")
with st.sidebar:
    api_key=st.text_input("请输入Tongyi API
Key: ", type="password")
    st.markdown("[获取Tongyi API
key](https://bailian.console.aliyun.com/?ap
iKey=1#/api-key)")
    if "memory" not in st.session_state:
        st.session_state["memory"]=
ConversationBufferMemory(return_messages=Tr
ue)

st.session_state["messages"]=[{"role": "ai",
    "content": "你好，我是你的AI助手，有什么可以帮
你的吗？"}]
```

```
for message in st.session_state["messages"]:
    st.chat_message(message["role"]).write(message
["content"])
    prompt=st.chat_input()
    if prompt:
        if not api_key:
            st.info("请输入你的OpenAI API Key")
            st.stop()

        st.session_state["messages"].append({"role": "h
uman", "content": prompt})
        st.chat_message("human").write(prompt)
        with st.spinner("AI正在思考中，请稍等..."):
            response=
get_chat_response(prompt, st.session_state["mem
ory"], api_key)
            msg={"role": "ai", "content": response}
            st.session_state["messages"].append(msg)
            st.chat_message("ai").write(response)
```



传智教育旗下高端IT教育品牌